API Reference Documentation

SentimentIQ · Predictive Analytics Engine · v2.4.1

Document Type	Domain	Audience	Last Updated
REST API Reference	**AI / NLP / ML**	**ML Engineers & Devs**	**June 2026**

1. Overview

SentimentIQ is a machine learning-powered REST API that provides real-time sentiment analysis, intent classification, and emotion detection for unstructured text data. Built on a transformer-based architecture (fine-tuned BERT and RoBERTa models), SentimentIQ is designed for high-throughput production environments and supports multilingual inputs across 42 languages.

This reference document covers all public API endpoints, authentication flows, request/response schemas, error handling patterns, and usage best practices for integrating SentimentIQ into your applications.

1.1 Key Capabilities

Capability	Description
Sentiment Classification	Classifies text as Positive, Negative, or Neutral with confidence scores.
Emotion Detection	Identifies 27 fine-grained emotions (e.g., joy, frustration, anticipation).
Intent Recognition	Categorizes user intent across 150+ pre-trained intent classes.
Aspect-Based Sentiment	Extracts sentiment for specific aspects or entities within a document.
Batch Inference	Processes up to 500 texts per request for high-volume pipelines.
Streaming Responses	Server-sent events (SSE) for real-time analysis of long-form content.

1.2 Base URL

```
https://api.sentimentiq.ai/v2
```

All endpoints in this document are relative to the base URL above. The API uses HTTPS exclusively; HTTP requests are automatically redirected.

2. Authentication

SentimentIQ uses API key authentication combined with optional JWT bearer tokens for enhanced security in enterprise environments. All requests must include a valid API key in the request header.

2.1 API Key Authentication

Include your API key in the Authorization header using the Bearer scheme:

```
Authorization: Bearer sk-sentiq-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

2.2 Request Headers

Header	Required	Description
Authorization	Required	Bearer {API_KEY} — your SentimentIQ API key.
Content-Type	Required	Must be application/json for all POST requests.
X-Request-ID	Optional	Client-generated UUID for request tracing and idempotency.
X-Model-Version	Optional	Pin to a specific model version (e.g., bert-v3.1).

2.3 Rate Limits

Rate limits are enforced per API key and vary by subscription tier:

Tier	Requests/min	Texts/day	Batch Size	Concurrency
Starter	60	10,000	10	2
Growth	300	500,000	100	10
Enterprise	2,000	Unlimited	500	50

Note: Rate limit headers (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset) are included in every API response.

3. Endpoints

3.1 Analyze Sentiment

POST `/analyze/sentiment`

Performs sentiment classification on one or more input texts. Returns a confidence-weighted prediction for Positive, Negative, or Neutral sentiment, along with a composite sentiment score.

Request Body

Parameter	Type	Required	Description
texts	array[string]	Yes	Array of input texts to analyze. Max 500 items per request; each text ≤ 10,000 characters.
language	string	No	ISO 639-1 language code (e.g., en, es, fr). Defaults to auto-detect.
model	string	No	Model variant: standard (default), finance, healthcare, social-media.
granularity	string	No	Response detail level: summary (default) or detailed.
return_spans	boolean	No	If true, highlights sentiment-bearing spans in the text. Default: false.

Example Request

```
POST /analyze/sentiment
Content-Type: application/json
Authorization: Bearer sk-sentiq-xxxxxxxxxxxx

{
  "texts": [
    "The new model inference speed is outstanding!",
    "Latency increased by 40% after the last deployment."
  ],
  "model": "standard",
  "granularity": "detailed",
  "return_spans": true
}
```

Example Response


```
{
  "request_id": "req_01HZ9K3MN4P7QR8ST",
  "model_version": "bert-sentiment-v3.2",
  "processing_time_ms": 83,
  "results": [
    {
      "index": 0,
      "sentiment": "POSITIVE",
      "score": 0.9412,
      "confidence": {
        "positive": 0.9412,
        "neutral":  0.0441,
        "negative": 0.0147
      },
      "spans": [
        { "text": "outstanding", "start": 40, "end": 51, "sentiment": "POSITIVE",
"score": 0.98 }
      ]
    },
    {
      "index": 1,
      "sentiment": "NEGATIVE",
      "score": 0.8873,
      "confidence": {
        "positive": 0.0312,
```

```
        "neutral":  0.0815,
        "negative": 0.8873
      },
      "spans": [
        { "text": "increased by 40%", "start": 8, "end": 24, "sentiment":
"NEGATIVE", "score": 0.91 }
      ]
    }
  ]
}
```


3.2 Detect Emotions

POST	`/analyze/emotions`

Returns a probability distribution across 27 fine-grained emotion categories for each input text, based on the Plutchik emotion taxonomy extended with domain-specific classes.

Request Body

Parameter	Type	Required	Description
texts	array[string]	Yes	Input texts. Max 100 items; each text ≤ 5,000 characters.
top_n	integer	No	Return only top-N emotions by score. Range: 1–27. Default: 5.
threshold	float	No	Minimum confidence threshold to include an emotion (0.0–1.0). Default: 0.05.
include_taxonomy	boolean	No	Attach Plutchik taxonomy metadata to each result. Default: false.

Example Request

```
POST /analyze/emotions
Content-Type: application/json
Authorization: Bearer sk-sentiq-xxxxxxxxxxx

{
  "texts": ["I can't believe they shipped this without testing. Absolutely
furious."],
  "top_n": 3,
  "threshold": 0.1
}
```

Example Response


```
{
  "request_id": "req_02KA1L5NO8S2UV3WX",
  "model_version": "emotion-roberta-v2.1",
  "results": [
    {
      "index": 0,
      "primary_emotion": "anger",
      "emotions": [
        { "label": "anger",       "score": 0.8841, "category": "primary" },
```

```
        { "label": "disgust",     "score": 0.6712, "category": "primary" },
        { "label": "disapproval", "score": 0.4380, "category": "secondary" }
      ]
    }
  ]
}
```


3.3 Batch Analysis

POST `/analyze/batch`

Asynchronously processes large text corpora (up to 500 texts per request). Returns a job ID; use the job status endpoint to poll for results or configure a webhook for callbacks.

Request Body

Parameter	Type	Required	Description
texts	array[string]	Yes	Up to 500 input texts per batch request.
analyses	array[string]	Yes	Analysis types to run: sentiment, emotions, intent, aspects.
webhook_url	string	No	HTTPS URL to POST results when job completes. Signed with HMAC-SHA256.
priority	string	No	Job priority: normal (default) or high (Enterprise tier only).

3.4 Get Batch Job Status

GET `/jobs/{job_id}`

Retrieves the current status and results of a batch analysis job. Poll this endpoint until the status field is completed or failed.

Path Parameters

Parameter	Type	Description
job_id	string	The unique job identifier returned by POST /analyze/batch.

Response: Job Object


```
{
  "job_id": "job_3XY7MN9KL2PQ",
  "status": "completed",          // pending | processing | completed | failed
  "created_at": "2026-06-22T08:30:00Z",
  "completed_at": "2026-06-22T08:30:47Z",
  "progress": {
    "total": 500,
    "processed": 500,
    "failed": 0
  },
```

```
  "results_url": "https://api.sentimentiq.ai/v2/jobs/job_3XY7MN9KL2PQ/results"
}
```

4. Error Handling

The SentimentIQ API uses standard HTTP status codes. All error responses follow a consistent JSON schema that includes a machine-readable error code, a human-readable message, and contextual metadata to aid debugging.

4.1 Error Response Schema

```
{
  "error": {
    "code":       "VALIDATION_ERROR",
    "message":    "texts[2] exceeds the maximum length of 10,000 characters.",
    "request_id": "req_09BC2D4EF7GH5IJ6",
    "details": {
      "field":     "texts[2]",
      "limit":     10000,
      "received":  12483
    }
  }
}
```

4.2 HTTP Status Codes

Code	Error Code	Description
400	VALIDATION_ERROR	Request body failed schema validation. Check the details field.
401	UNAUTHORIZED	Missing or invalid API key.
403	FORBIDDEN	Valid key but insufficient permissions for this endpoint or model.
413	PAYLOAD_TOO_LARGE	Batch exceeds 500 texts or individual text exceeds character limit.
422	UNSUPPORTED_LANGUAGE	Detected language is not supported by the selected model.
429	RATE_LIMIT_EXCEEDED	Too many requests. Retry after X-RateLimit-Reset seconds.
500	INTERNAL_ERROR	Unexpected server error. Retry with exponential backoff.
503	MODEL_UNAVAILABLE	Model is being updated. Retry after 30–60 seconds.

4.3 Retry Strategy

For transient errors (429, 503, 500), implement exponential backoff with jitter:

- Start with a 1-second delay on the first retry.
- Double the delay with each subsequent attempt (1s → 2s → 4s → 8s).
- Add ±25% jitter to prevent thundering herd situations.
- Set a maximum retry count of 5 and a maximum delay cap of 60 seconds.
- Do not retry 4xx errors except for 429 Rate Limit responses.

5. SDKs & Code Samples

Official SDKs are available for the following languages. All SDKs are open-source, follow semantic versioning, and include built-in retry logic, type definitions, and async support.

Language	Package	Version	Install
Python	sentimentiq-python	2.4.1	pip install sentimentiq
Node.js / TypeScript	sentimentiq-node	2.4.1	npm install sentimentiq
Go	go-sentimentiq	2.4.1	go get github.com/sentiq/go-sentiq
Java	sentimentiq-java	2.4.1	Maven / Gradle artifact

5.1 Python Example

```
from sentimentiq import SentimentIQ

client = SentimentIQ(api_key="sk-sentiq-xxxxxxxxxxxx")

# Single-document sentiment analysis
result = client.sentiment.analyze(
    texts=["Our model accuracy improved by 12% this quarter!"],
    model="standard",
    granularity="detailed",
)

print(result.results[0].sentiment)    # POSITIVE
print(result.results[0].score)        # 0.9621
```

5.2 Node.js / TypeScript Example

```
import SentimentIQ from "sentimentiq";

const client = new SentimentIQ({ apiKey: process.env.SENTIQ_API_KEY });

const result = await client.emotions.detect({
  texts: ["I'm genuinely excited about this release!"],
  topN: 3,
  threshold: 0.1,
});

console.log(result.results[0].primaryEmotion); // joy
```

```
console.log(result.results[0].emotions);        // [{label: "joy", score: 0.94},
...]
```

6. ML Model Details

Understanding the underlying models helps you select the right configuration and correctly interpret outputs.

6.1 Available Models

Model Variant	Architecture	F1 Score	Best For
standard	RoBERTa-large	0.934	General-purpose text; news, reviews, support.
finance	FinBERT-v2	0.961	Earnings calls, financial reports, SEC filings.
healthcare	BioBERT-v1.2	0.948	Clinical notes, patient feedback, EHR data.
social-media	TweetEval-XL	0.902	Tweets, Reddit posts, short-form UGC.

6.2 Interpreting Confidence Scores

Each prediction includes a confidence score in the range [0.0, 1.0]. Use these thresholds as guidelines:

Score Range	Reliability	Recommended Action
≥ 0.90	High	Use prediction directly in automated workflows.
0.70 – 0.89	Medium	Suitable for most use cases; flag edge cases for review.
0.50 – 0.69	Low	Route to human review or apply a fallback strategy.
< 0.50	Uncertain	Discard prediction or treat as neutral.

6.3 Known Limitations

- Sarcasm and irony reduce accuracy by up to 15% in informal text; use social-media model variant for best results.
- Texts shorter than 10 tokens may produce unreliable confidence scores due to insufficient context.
- Domain-specific jargon outside of the pre-trained vocabulary may be silently tokenized as [UNK], reducing accuracy.
- Multilingual inputs with code-switching (mixing languages mid-sentence) should be pre-segmented before submitting.

7. Changelog

Version	Date	Changes
v2.4.1	Jun 2026	Improved tokenization for code-switched multilingual inputs; patched edge case in aspect extraction for texts > 5,000 chars.
v2.4.0	Apr 2026	Added streaming SSE endpoint for real-time analysis. Introduced healthcare model variant (BioBERT-v1.2).
v2.3.0	Jan 2026	Batch job prioritization for Enterprise tier. HMAC-SHA256 webhook signing. Java SDK GA release.
v2.2.0	Oct 2025	Emotion detection API GA. 27-class taxonomy replacing the previous 8-class model.
v2.0.0	Jul 2025	Major release: RoBERTa-large upgrade, aspect-based sentiment, new error schema, rate limit headers.